



Smart Tourism Platform for Sharm El Sheikh

Graduation Project Documentation

Prepared by

Team Member	Role	Responsibilities
Seif Waleed Shalaby (Team Leader)	Frontend Developer & AI Feature Lead	Team coordination, AI Vacation Planner module, feature planning, integration, project management, APIs & database design.
Youssef Osama Samir	Frontend & Backend Developer	Explore Page development and related UI components.
Abd Alrahman Abdallah Madawy	Frontend & Backend Developer	Home Page and My Account module implementation.
Yasmin Abdelaziz Mohamed	Frontend & Backend Developer	Authentication (Login & Signup) and Ride Booking module.
Islam Nasser Ahmed	Frontend & Backend Developer	Trips and Booking module implementation.

Track: Frond End Using (React) — Digital Egypt Pioneers Initiative (DEPI)

Supervisor: (Mohammed Emad)

Submission Date: (7/2026)

Repository: <https://github.com/Seif-Waleed2/go-sharm>

Table of Contents

1. Project Planning & Management	3
1.1 Project Proposal	3
1.2 Project Plan (Timeline)	3
1.3 Task Assignment & Roles	3
1.4 Risk Assessment & Mitigation Plan	4
1.5 KPIs (Key Performance Indicators)	4
2. Literature Review	5
2.1 Feedback & Evaluation	5
2.2 Suggested Improvements	5
2.3 Final Grading Criteria	5
3. Requirements Gathering	6
3.1 Stakeholder Analysis	6
3.2 User Stories & Use Cases	6
3.3 Functional Requirements	6
3.4 Non-Functional Requirements	7
4. System Analysis & Design	8
4.1 Problem Statement & Objectives	8
4.2 Database Design & Data Modeling	10
4.3 Data Flow & System Behavior	13
4.4 UI/UX Design & Prototyping	14
4.5 System Deployment & Integration	18
4.6 Additional Deliverables	18
5. Implementation (Source Code & Execution)	20
5.1 Source Code	20
5.2 Version Control & Collaboration	21
5.3 Deployment & Execution	21
6. Testing & Quality Assurance	22
Test Cases & Test Plan	22
Automated Testing	23
Bug Reports	23
7. Final Presentation & Reports	25
User Manual	25
Technical Documentation	25
Project Presentation	25
Video Demonstration (Optional)	25

1. Project Planning & Management

1.1 Project Proposal

Go Sharm is a smart tourism platform developed to improve the travel experience in Sharm El Sheikh. The application combines AI-powered vacation planning, ride booking, destination exploration, and trip management into a single web application. The platform allows tourists to discover attractions, generate personalized travel plans based on their interests, book transportation, and manage their trips through an easy-to-use interface.

Objectives: Simplify tourism planning; help tourists discover attractions; generate AI-powered travel plans; provide ride booking; manage bookings and trip history; deliver a modern, responsive user experience.

Scope: A single-role (Tourist) web application covering authentication, place discovery, AI itinerary generation, ride booking, and trip/profile management. Administrative and provider dashboards, payment gateways, and live chat are explicitly out of scope for this release.

1.2 Project Plan (Timeline)

High-level milestone timeline for the project (dates to be finalized by the team):

Phase	Milestone	Target Date
Planning	Requirements gathering & UI reference screens finalized	[4/2026]
Design	Database schema, API contract, and wireframe review complete	[5/2026]
Backend	Express MVC backend (auth, places, rides, trips, AI) implemented	[6/2026]
Frontend	React pages implemented matching UI designs	[6/2026]
Integration	Frontend connected to backend; MongoDB Atlas connectivity resolved	[6/2026]
Testing	Manual QA pass across all modules	[7/2026]
Submission	Documentation, presentation, and final repository push	[7/2026]

(Note: replace bracketed placeholders with the team's actual Gantt-chart dates before final submission.)

1.3 Task Assignment & Roles

Team Member	Role	Responsibilities
Seif Waleed Shalaby (Team Leader)	Frontend Developer & AI Feature Lead	Team coordination, AI Vacation Planner module, feature planning, integration, project management, APIs & database design.
Youssef Osama Samir	Frontend & Backend Developer	Explore Page development and related UI components.

Team Member	Role	Responsibilities
Abd Alrahman Abdallah Madawy	Frontend & Backend Developer	Home Page and My Account module implementation.
Yasmin Abdelaziz Mohamed	Frontend & Backend Developer	Authentication (Login & Signup) and Ride Booking module.
Islam Nasser Ahmed	Frontend & Backend Developer	Trips and Booking module implementation.

1.4 Risk Assessment & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Database connectivity failures (e.g. DNS/SRV resolution to MongoDB Atlas)	Medium	High	Documented root-cause debugging process; fallback to standard (non-SRV) connection string; verify Network Access IP allow-list.
Scope creep beyond the approved feature list	Medium	Medium	PRD explicitly excludes admin dashboards, payments, and chat; changes require team lead sign-off.
Uneven module integration between 5 contributors	Medium	Medium	Shared MVC backend structure and a single REST API contract agreed up front; feature branches per module.
AI itinerary results feeling generic	Low	Medium	Rule-based matching against seeded place categories, expandable later to a real LLM-backed service.
Missing environment configuration on new machines	Medium	Low	.env.example files provided for both client and server with a documented setup guide in the README.

1.5 KPIs (Key Performance Indicators)

- API response time: target under 300ms for standard CRUD endpoints on local/staging environments.
- System uptime: target 99% during the demo/evaluation period once deployed.
- User adoption (demo cohort): percentage of test users able to complete registration, an AI plan, and a ride booking without assistance.
- Task completion rate: percentage of core user journeys (sign up → explore → plan → book) completed without errors during manual QA.
- Bug resolution time: average time between a bug being identified and a verified fix (see Section 6 for real examples).

2. Literature Review

2.1 Feedback & Evaluation

[Reserved for supervisor/lecturer assessment. To be completed after project review — include strengths observed, weaknesses identified, and overall evaluation notes.]

2.2 Suggested Improvements

Areas identified by the team as opportunities for future enhancement:

- Replace the rule-based AI itinerary engine with a real LLM-backed recommendation service for richer, more personalized plans.
- Add automated test coverage (unit tests for controllers/services, integration tests for API routes) — current QA is manual only.
- Introduce a payments flow for ride bookings and paid attractions (explicitly out of scope for this release).
- Add a provider/admin dashboard for managing places and monitoring bookings (explicitly out of scope for this release).
- Integrate a real mapping/geocoding provider for accurate ride distance and fare calculation, replacing the current simplified estimate.
- Add CI/CD pipelines for automated build/test/deploy on every push (see Section 5.2).

2.3 Final Grading Criteria

Suggested breakdown for evaluating this submission, aligned with the documentation guideline sections:

Criterion	Weight	Covered In
Documentation completeness & clarity	20%	This document, Sections 1–7
System analysis & design quality	20%	Section 4
Implementation & code quality	25%	Section 5, GitHub repository
Testing & QA	15%	Section 6
UI/UX & functionality	15%	Section 4.4, live demo
Presentation & delivery	5%	Section 7

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Type	Interest / Need
Tourists visiting Sharm El Sheikh	Primary	Discover attractions, plan trips, book rides, manage bookings in one app.
Ride drivers / transport operators	Secondary	Receive ride requests and trip details through a consistent booking flow.
Local businesses (restaurants, dive centers, beaches, etc.)	Secondary	Increased visibility to tourists through the Explore module and AI itinerary.
Development team	Internal	Clear requirements, maintainable architecture, gradeable deliverable.
Course supervisor / evaluators	Internal	A working, well-documented system that meets the stated objectives.

3.2 User Stories & Use Cases

Representative user stories, written from the Tourist's perspective:

1. As a visitor, I want to create an account so that I can save my trips and preferences.
2. As a tourist, I want to browse places by category (beaches, restaurants, safaris, etc.) so I can find things that interest me.
3. As a tourist, I want to search for a specific place or activity so I don't have to scroll through everything.
4. As a tourist, I want an AI-generated day-by-day itinerary based on my trip dates, trip type, and interests, so I don't have to plan manually.
5. As a tourist, I want to request a ride between two locations and see an estimated fare before booking.
6. As a tourist, I want to view my past trips and rebook a previous ride quickly.
7. As a tourist, I want to edit my profile information.

See Section 4.1 for the corresponding Use Case diagram.

3.3 Functional Requirements

- The system shall allow a user to register with name, email, and password.
- The system shall hash passwords before storing them and never return password data in API responses.
- The system shall authenticate users and issue a JWT on successful login.
- The system shall allow authenticated users to view and update their profile.
- The system shall list places, and support filtering by category and free-text search.
- The system shall generate a day-by-day AI itinerary from a date range, trip type, and selected interests, and persist it against the user.
- The system shall calculate an estimated ride fare from pickup/dropoff locations and ride type.
- The system shall allow authenticated users to book a ride, storing it as a trip record.
- The system shall allow authenticated users to view their trip history.

3.4 Non-Functional Requirements

- Security: passwords hashed with bcrypt; protected routes require a valid JWT; all inputs validated server-side with express-validator.
- Performance: API endpoints should respond within ~300ms under normal local/staging load.
- Usability: responsive layout usable on both desktop and mobile viewports; light and dark themes supported.
- Maintainability: backend follows an MVC structure with clearly separated routes, controllers, services, models, and middleware.
- Reliability: centralized error handling returns consistent, predictable JSON error responses across all endpoints.
- Portability: environment-specific configuration (DB URI, JWT secret, API base URL) is isolated to `.env` files, never hard-coded.

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem: Tourists visiting Sharm El Sheikh often need multiple applications to find attractions, plan their vacation, book transportation, save favorite places, and manage bookings. Switching between multiple services creates a poor user experience.

Goal: Go Sharm integrates discovery, AI-assisted planning, transportation, and trip management into a single platform, so a tourist never has to leave the app to complete a travel task.

Objectives:

- Simplify tourism planning for visitors.
- Help tourists discover attractions relevant to their interests.
- Generate AI-powered, personalized travel plans.
- Provide an in-app ride booking experience with transparent pricing.
- Let users manage bookings and review trip history.
- Deliver a modern, responsive user experience across devices.

Use Case Diagram

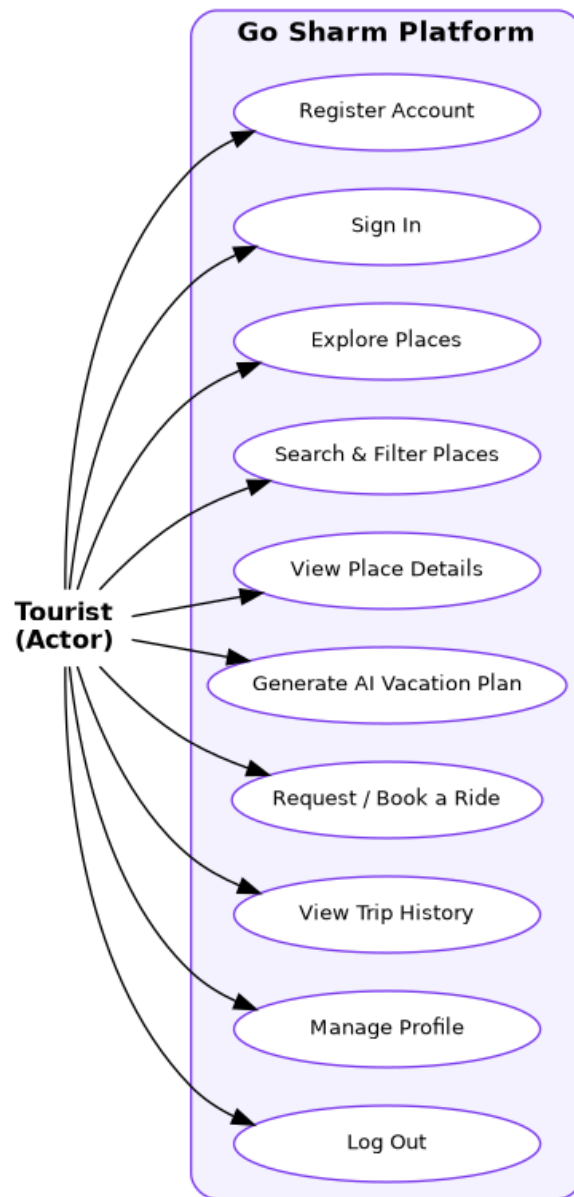


Figure 4.1 — Use Case Diagram: Tourist actor and system use cases.

Use case descriptions:

Use Case	Description	Auth Required
Register Account	Create a new tourist account with name, email, password.	No
Sign In	Authenticate with email/password and receive a JWT.	No
Explore Places	Browse tourist attractions, filterable by category.	No
Search & Filter Places	Free-text search combined with category filter.	No
View Place Details	View full details of a single place.	No

Use Case	Description	Auth Required
Generate AI Vacation Plan	Submit dates, trip type, interests; receive a saved day-by-day itinerary.	Yes
Request / Book a Ride	Enter pickup/dropoff, view fare options, confirm a booking.	Yes (to confirm)
View Trip History	View past ride bookings with details and rebook option.	Yes
Manage Profile	View and edit account information.	Yes
Log Out	End the authenticated session.	Yes

Software Architecture

The backend follows the MVC (Model-View-Controller) pattern, adapted for a REST API (no server-rendered views): Routes define endpoints and attach validation; Controllers hold request/response logic; Services encapsulate business logic that doesn't belong in a controller (e.g. the AI itinerary engine); Models define Mongoose schemas; Middleware handles cross-cutting concerns (JWT auth, validation, centralized error handling). The frontend is a component-based React SPA communicating with the API exclusively over JSON/REST.

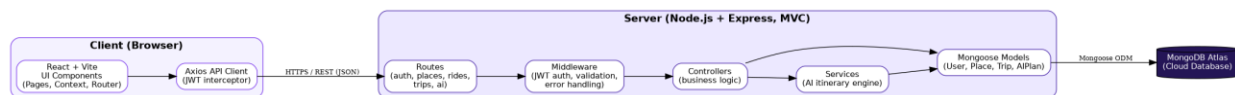


Figure 4.2 — High-level component architecture.

4.2 Database Design & Data Modeling

Go Sharm uses MongoDB with Mongoose schemas. Four core collections model the system: Users, Places, Trips, and AIPlans.

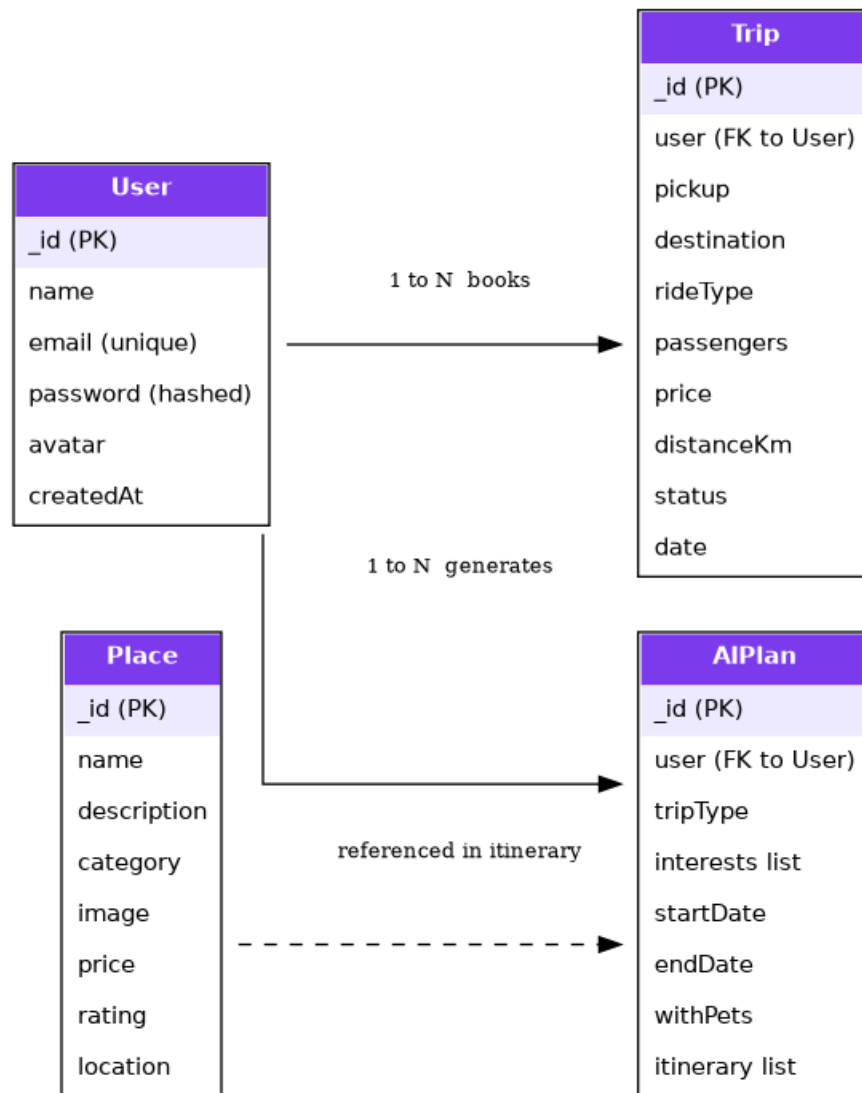


Figure 4.3 — Entity-Relationship Diagram.

Logical schema — User

Field	Type	Constraints
name	String	required, 2–60 chars
email	String	required, unique, lowercase, validated format
password	String	required, min 6 chars, hashed with bcrypt, select: false
avatar	String	optional URL
createdAt / updatedAt	Date	auto (timestamps)

Logical schema — Place

Field	Type	Constraints
name	String	required

Field	Type	Constraints
description	String	optional
category	String (enum)	Beach, Restaurant, Safari Excursion, Bazar, Yacht Excursions, Diving Centers, Bank, Entertainment, Shopping, Mountain Lounge
image	String	URL
price	Number	required, min 0
rating	Number	0–5
location	String	optional

Logical schema — Trip

Field	Type	Constraints
user	ObjectId (ref User)	required
pickup / destination	String	required
rideType	String (enum)	Smart X, Smart XL, Smart VIP
passengers	Number	min 1, default 1
price / distanceKm	Number	min 0
status	String (enum)	pending, confirmed, completed, cancelled
date	Date	default now

Logical schema — AIPlan

Field	Type	Constraints
user	ObjectId (ref User)	required
tripType	String (enum)	Solo, Partner Trip, Family Trip, Friends Trip
interests	String[]	e.g. Sea View, Desert Safari, Scuba Diving
startDate / endDate	Date	required
withPets	Boolean	default false
itinerary	Array of { day, title, activities[] }	generated by aiService.js

4.3 Data Flow & System Behavior

Data Flow Diagram (Context Level)

At context level, the Tourist is the sole external entity. Requests flow: Tourist → [Go Sharm System] → MongoDB Atlas, with responses flowing back through the same path. Level 1 decomposes the system process into: 1) Auth Process, 2) Explore/Places Process, 3) AI Planning Process, 4) Ride Booking Process, 5) Trip History Process — each reading/writing its corresponding data store (Users, Places, Trips, AIPlans).

Interaction Flow — AI Vacation Plan Generation



Figure 4.4 — Interaction flow for POST /api/ai/generate.

Activity Diagram — Ride Booking Flow

Described as a sequential activity flow (textual, in lieu of a graphical activity diagram):

- 1. Tourist enters pickup and dropoff locations.
- 2. System calls GET /api/rides and calculates fares for Smart X / Smart XL / Smart VIP.
- 3. Tourist selects a ride type and payment method.
- 4. System checks authentication — if not signed in, tourist is redirected to Sign In and returned afterward.
- 5. Tourist confirms booking; system calls POST /api/rides.
- 6. System creates a Trip record with status "confirmed" and returns a confirmation.
- 7. Tourist is redirected to the Trips page, where the new booking appears.

State Diagram — Trip Status

A Trip moves through the following states (Trip.status field):

- pending → initial state if a booking requires confirmation before dispatch.
- confirmed → default state set immediately on booking in the current implementation.
- completed → set once the ride has been carried out (future enhancement: driver-side status update).
- cancelled → set if the tourist or system cancels the booking (future enhancement: cancellation endpoint).

Class Diagram (Simplified — Backend Models)

The backend's "classes" are its Mongoose models, each with schema-defined attributes and instance/static behavior:

- User: fields as in Section 4.2; method comparePassword(candidate) → Boolean; pre-save hook hashes password.
- Place: fields as in Section 4.2; text index across name/description/category for search.
- Trip: fields as in Section 4.2; created by rideController.bookRide and tripController.createTrip.
- AIPlan: fields as in Section 4.2; itinerary populated by aiService.generateItinerary().

4.4 UI/UX Design & Prototyping

Screens were implemented in React + Tailwind CSS to match the approved reference designs. Representative screens from the working application:

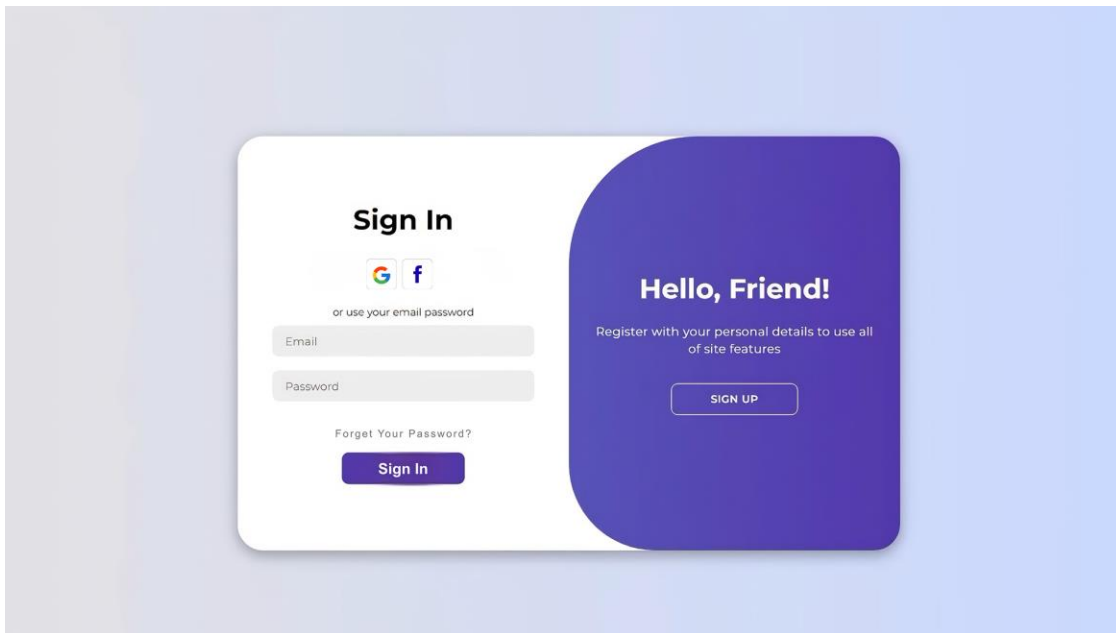


Figure 4.5 — Sign In screen.

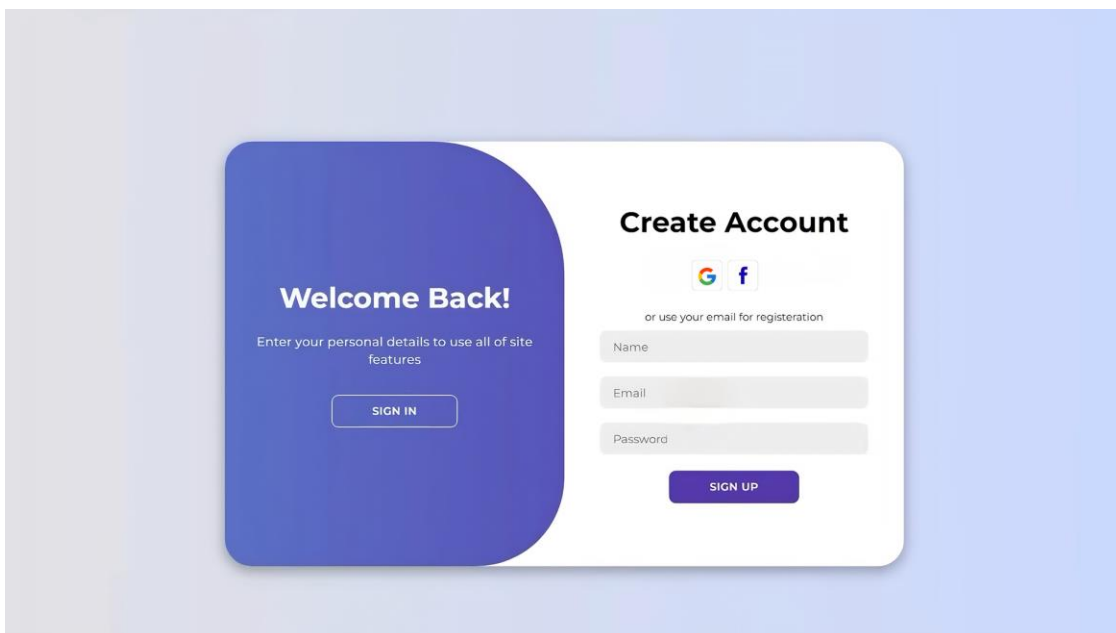


Figure 4.6 — Create Account screen.

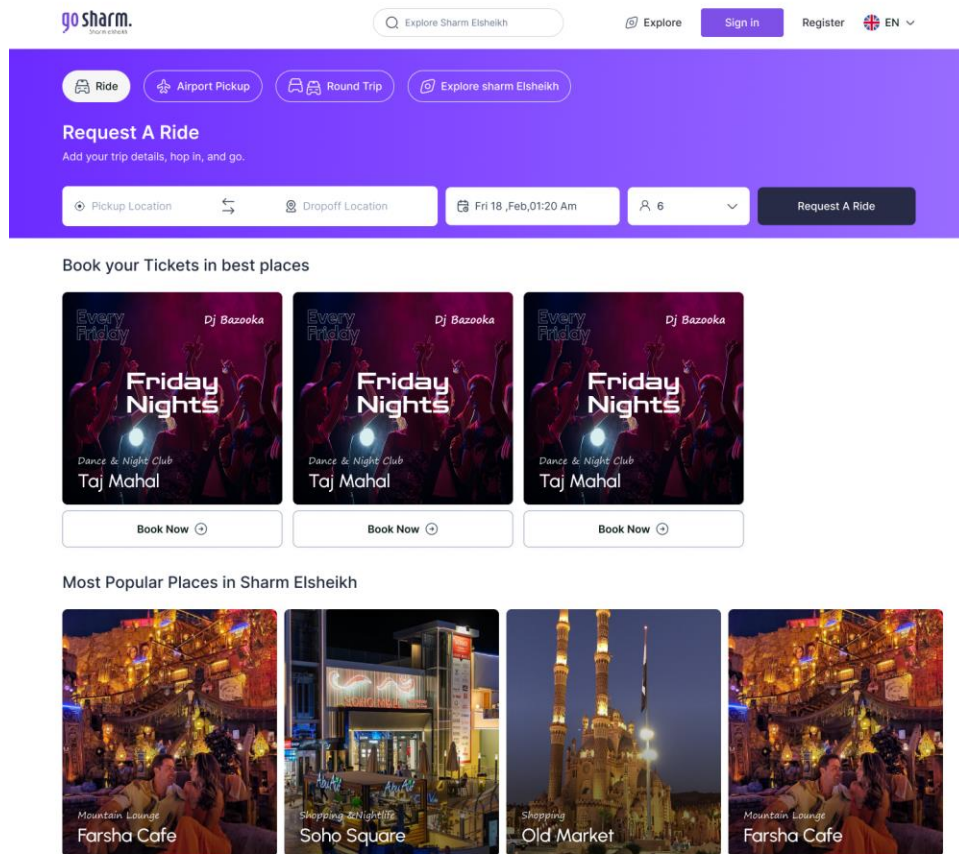


Figure 4.7 — Home screen (Ride tab).

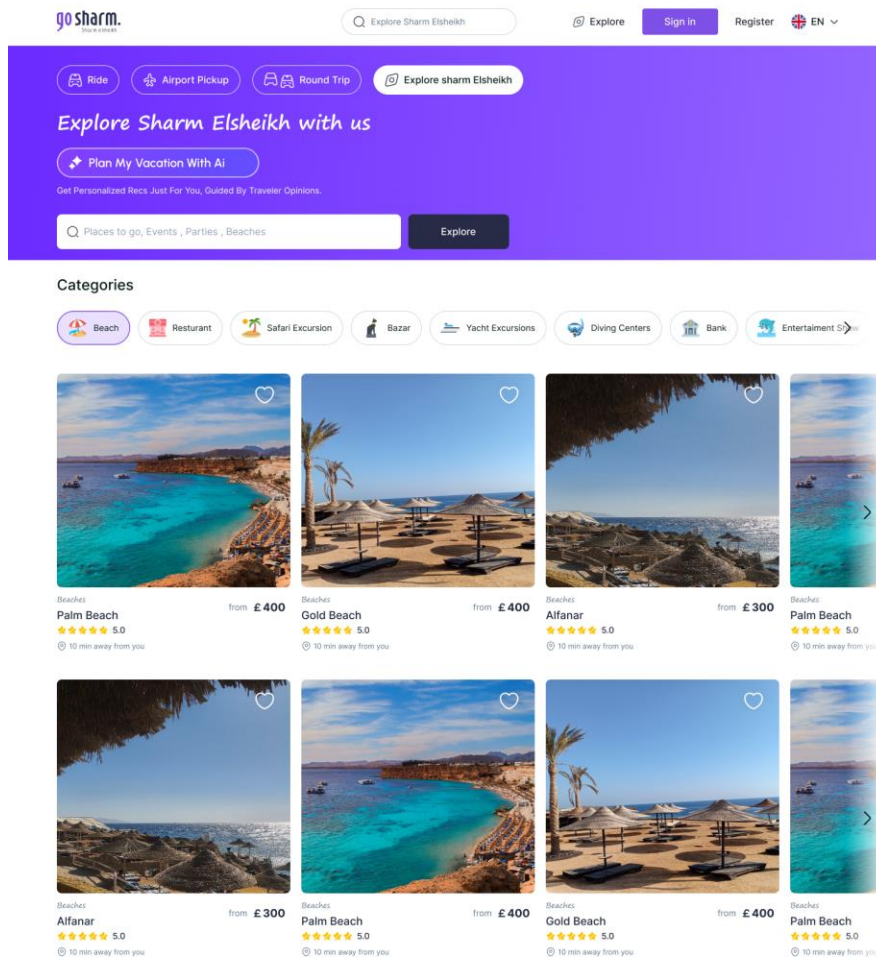


Figure 4.8 — Explore screen with category filters and place cards.

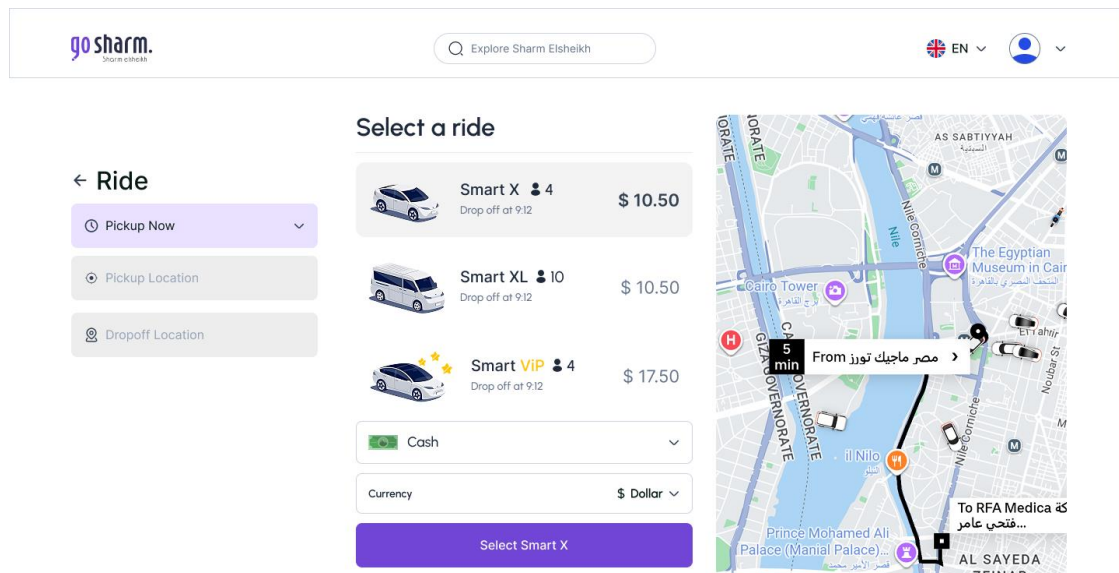


Figure 4.9 — Ride booking screen with fare comparison.

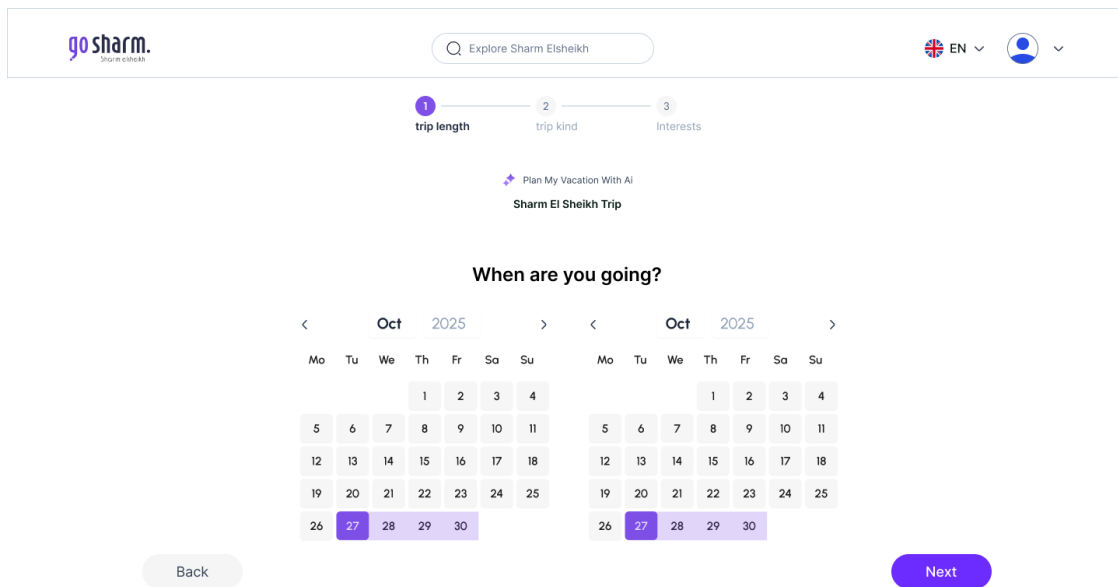


Figure 4.10 — AI Vacation Planner (interests step).

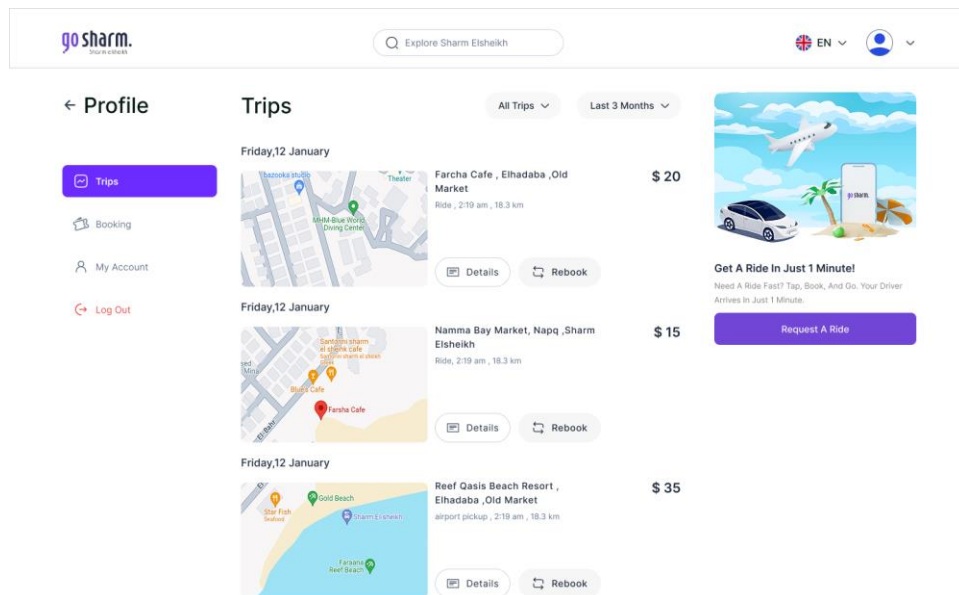


Figure 4.11 — Trips / Profile screen.

UI/UX Guidelines

- Color palette: primary brand purple (#7C53FF family, from #F4F2FF tint to #241556 shade), used for CTAs, active states, and gradients.
- Typography: Poppins for all UI text, with clear weight hierarchy (extrabold headings, medium body).
- Components: rounded-corner cards and pills (rounded-2xl / rounded-full) throughout, consistent with the reference screens.
- Accessibility: sufficient color contrast on interactive elements; form inputs paired with visible placeholder text and inline validation errors.
- Responsiveness: mobile-first Tailwind breakpoints; navigation collapses gracefully on small viewports.

- Theming: light theme by default (matching the reference designs), with an optional dark theme toggle in the navbar.

4.5 System Deployment & Integration

Technology Stack

Layer	Technology
Frontend	React 19, Vite, Tailwind CSS, React Router, Axios, react-hot-toast, lucide-react
Backend	Node.js, Express.js (MVC architecture)
Database	MongoDB Atlas, Mongoose ODM
Authentication	JWT (jsonwebtoken), bcryptjs password hashing
Validation	express-validator
Tooling	nodemon (dev), morgan (logging), cors

Deployment Diagram

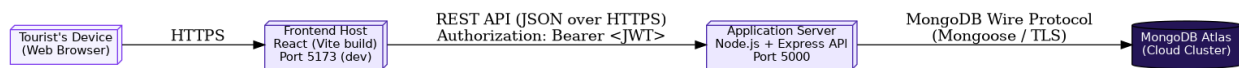


Figure 4.12 — Deployment diagram: browser, frontend host, API server, and MongoDB Atlas.

Component Diagram

See Figure 4.2 in Section 4.1 for the full component breakdown of client and server layers.

4.6 Additional Deliverables

API Documentation

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Create an account; returns JWT + user.
POST	/api/auth/login	No	Authenticate; returns JWT + user.
GET	/api/auth/profile	Yes	Get the current user's profile.
PUT	/api/auth/profile	Yes	Update name/avatar.
GET	/api/places	No	List places (?search=, ?category=).
GET	/api/places/:id	No	Get a single place.
GET	/api/rides	No	Get ride type options + estimated price for a route.
POST	/api/rides	Yes	Book a ride (creates a Trip).

Method	Endpoint	Auth	Description
GET	/api/trips	Yes	Get the user's trip history.
GET	/api/trips/:id	Yes	Get a single trip.
POST	/api/trips	Yes	Manually create/rebook a trip.
POST	/api/ai/generate	Yes	Generate & save an AI vacation itinerary.
GET	/api/ai/plans	Yes	List the user's saved AI plans.

Testing & Validation

See Section 6 for the full test plan and bug reports.

Deployment Strategy

The application currently runs locally (frontend on Vite's dev server, backend on Express, database on MongoDB Atlas). Recommended path to a live deployment: host the frontend as a static build on Vercel or Netlify; host the backend on Render or Railway; keep MongoDB Atlas as the managed database with IP access rules relaxed to allow the hosting provider's egress IPs (or 0.0.0.0/0 for simplicity during evaluation). CORS `CLIENT\_URL` and the frontend's `VITE\_API\_URL` must be updated to the deployed URLs.

5. Implementation (Source Code & Execution)

5.1 Source Code

Structured & Well-Commented Code

The codebase is organized as a two-package repository (client/ and server/). Every controller, service, middleware, and model file carries a short JSDoc-style comment describing its purpose, and route files group related endpoints with inline validation rules so the request contract is visible at the route definition itself.

Coding Standards & Naming Conventions

- camelCase for variables and functions; PascalCase for React components and Mongoose models.
- Files named after the entity/feature they implement (e.g. tripController.js, RideBooking.jsx).
- Consistent async/await usage on the backend, wrapped in a shared asyncHandler utility instead of repeated try/catch blocks.
- Consistent Tailwind utility ordering and a shared component-class layer (btn-primary, input-field, card) to avoid style duplication.

Modular Code & Reusability

Backend (MVC):

- config/ — database connection.
- models/ — Mongoose schemas (User, Place, Trip, AIPlan).
- controllers/ — request handling & response shaping.
- routes/ — endpoint definitions + express-validator rules.
- middleware/ — JWT auth guard, validation runner, centralized error handler.
- services/ — AI itinerary generation logic, decoupled from the controller.
- utils/ — asyncHandler, ApiError, generateToken.

Frontend:

- api/ — one Axios module per resource (auth, places, rides, trips, ai) sharing a single configured Axios instance.
- context/ — AuthContext (session state) and ThemeContext (light/dark) shared app-wide via React Context.
- components/ — reusable UI (Navbar, PlaceCard, ProfileLayout, LoadingSpinner, ProtectedRoute, VacationCalendar).
- pages/ — one component per route, composed from the shared components above.

Security & Error Handling

- Passwords are hashed with bcrypt (10 salt rounds) in a Mongoose pre-save hook; the password field is excluded from query results by default (select: false).
- JWT-protected routes verify the token via a protect middleware, attaching the resolved user to req.user before the controller runs.

- All request bodies are validated with express-validator before reaching controller logic; invalid requests return 400 with a field-level error list.
- A centralized error middleware normalizes Mongoose CastError, duplicate-key, and ValidationError cases into consistent JSON error responses with appropriate HTTP status codes.
- CORS is restricted to the configured CLIENT_URL rather than left open to all origins.

5.2 Version Control & Collaboration

Repository: <https://github.com/Seif-Waleed2/go-sharm> (see link below).

<https://github.com/Seif-Waleed2/go-sharm>

Branching Strategy: [To be confirmed by the team — e.g. feature branches per module (feature/ai-planner, feature/ride-booking, feature/auth) merged into main via pull request, or trunk-based development with direct commits to main for this project’s scope.]

Commit History & Documentation: [Team to ensure commit messages follow a consistent convention, e.g. Conventional Commits (feat:, fix:, docs:, refactor:), and that pull requests include a short description of the change and testing performed.]

CI/CD Integration: Not yet implemented in this release. Recommended next step: a GitHub Actions workflow that runs npm install / npm run build for both client and server on every push, as a baseline continuous-integration check before adding automated tests (see Section 2.2).

5.3 Deployment & Execution

README File Summary

The repository README documents:

- Installation steps for both client/ and server/ (npm install in each).
- System requirements: Node.js, npm, and a MongoDB connection (local or Atlas).
- Configuration: .env.example provided for both packages (MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN, CLIENT_URL for the server; VITE_API_URL for the client).
- Execution guide: npm run seed then npm run dev for the backend; npm run dev for the frontend.
- A full REST API reference table (see Section 4.6).

Executable Files & Deployment Link

This project is a web application, not a compiled executable — there is no .exe/.jar/.apk artifact. Current status: running locally only (see Section 4.6, Deployment Strategy, for the recommended path to a public deployment link).

6. Testing & Quality Assurance

Test Cases & Test Plan

Testing was performed manually by exercising each feature end-to-end in the browser against the running backend and a seeded MongoDB Atlas database.

#	Test Scenario	Steps	Expected Result
TC-01	User registration	Fill name/email/password on Create Account, submit.	Account created, JWT stored, user redirected to Home while signed in.
TC-02	Duplicate email registration	Register again with the same email.	API returns 409 Conflict with a clear error message.
TC-03	User login	Enter valid credentials on Sign In.	JWT issued, user redirected to Home.
TC-04	Invalid login	Enter wrong password.	API returns 401; toast shows "Invalid email or password".
TC-05	Protected route without auth	Visit /trips or /account while signed out.	Redirected to Sign In; original destination preserved.
TC-06	Explore & filter places	Open Explore, switch category filters, search a keyword.	Grid updates to match category/search on each change.
TC-07	AI plan generation	Complete all 3 wizard steps and submit.	Loading screen shown, then a day-by-day itinerary rendered from seeded places.
TC-08	Ride fare calculation	Enter pickup & dropoff on Ride page.	Smart X / XL / VIP options appear with distance and price.
TC-09	Ride booking (signed in)	Select a ride type, confirm.	Trip created; redirected to Trips page showing the new booking.
TC-10	Ride booking (signed out)	Attempt to confirm a ride while signed out.	Toast prompts sign-in; redirected to Sign In.
TC-11	Profile update	Edit name on My Account, save.	Updated name persisted and reflected in the navbar/account page.

#	Test Scenario	Steps	Expected Result
TC-12	Logout	Click Log Out from navbar or sidebar.	Session cleared; protected pages become inaccessible.

Automated Testing

No automated test suite exists yet for this release — all testing above was performed manually. This is tracked as a suggested improvement (Section 2.2): unit tests for controllers/services (e.g. Jest + Supertest) and integration tests for the REST API.

Bug Reports

Two real issues were identified and resolved during integration testing:

Bug #1 — Empty data on Home/Explore, unable to log in or register

Field	Details
Symptom	Home and Explore pages rendered with no place data; Sign In / Register requests silently failed; ride booking did not work.
Root Cause	The backend server was not running / not connected to a MongoDB instance, so every API call failed. The frontend caught these failures and rendered empty states instead of surfacing an error.
Resolution	Verified backend health via GET /api/health and GET /api/places directly with curl before touching the frontend; confirmed MongoDB connection status in server logs; corrected the setup so MONGO_URI pointed at a reachable database.
Status	Resolved.

Bug #2 — MongoDB Atlas connection failure: querySrv ECONNREFUSED

Field	Details
Symptom	npm run dev / npm run seed failed immediately with Error: querySrv ECONNREFUSED _mongodb._tcp.<cluster>.mongodb.net, despite correct credentials, correct Network Access IP allow-list, and a working internet connection.
Root Cause	The mongodb+srv:// connection string requires the Node.js process to perform its own DNS SRV and TXT lookups before any connection is attempted. The machine's default DNS resolver (used by Node's c-ares-based dns module) was actively refusing that specific UDP query type, even though external tools like nslookup succeeded (nslookup uses a different resolution path with fallback behavior that Node's resolver does not have). This ruled out Atlas configuration, authentication, and Network Access as causes, since the failure occurred before any credentials or IP were even used.
Diagnosis Steps	Reproduced the failure directly with Node's dns.resolveSrv() (isolating it from Mongoose); confirmed the query succeeded when Node was pointed at a public

Field	Details
	resolver (8.8.8.8) instead of the default one — proving the local DNS path, not Atlas, was at fault.
Resolution	Replaced the SRV connection string with Atlas's standard (non-SRV) connection string, which only requires ordinary A-record lookups for the three shard hosts and a manually-supplied replicaSet name — removing the SRV/TXT lookup step entirely. No application code changes were required, only the MONGO_URI value in .env.
Status	Resolved.

7. Final Presentation & Reports

User Manual

Quick-start guide for a first-time tourist user:

8. Open the app and click Register to create an account (name, email, password), or Sign In if you already have one.
9. From Home, use the Explore sharm Elsheikh tab to browse categories, or the Ride tab to request transportation directly.
10. Click Plan My Vacation With AI to open the 3-step planner: choose your travel dates, choose your trip kind (Solo/Partner/Family/Friends) and whether you're bringing pets, then select your interests and submit.
11. Review your generated day-by-day itinerary on the result screen.
12. To book a ride, go to the Ride page, enter pickup and dropoff locations, choose a ride type, and confirm — you'll need to be signed in.
13. View past bookings anytime from Trips (accessible from the account menu), and rebook a previous ride in one click.
14. Update your name from My Account.

Technical Documentation

The complete technical documentation for Go Sharm is this document (Sections 1–6), covering system architecture, database schema, API reference, and testing. Source-level documentation lives alongside the code as inline comments in the GitHub repository.

Project Presentation

Status: [Placeholder — a PPT/PDF summarizing the problem, solution, architecture, demo screenshots, challenges faced (e.g. the MongoDB SRV DNS issue in Section 6), and outcomes should be prepared separately and linked/attached here before final submission.]

- Suggested structure: Problem → Solution Overview → Architecture → Live Demo → Challenges & Solutions → Results → Future Work.

Video Demonstration (Optional)

Status: [Placeholder — an optional short screen-recording walking through registration, exploring places, generating an AI plan, and booking a ride can be linked here.]